Using RBS

Having taken the effort to create these type annotations, what can we do with them? Well, there are basically two answers:

- There are some command-line tools that will do static analysis of your Ruby code. For example, based on the RBS files, a tool might find cases where the code does not match the type information, indicating a potential bug.
- Depending on the editor or development environment you are using, the tool may be able to use the RBS files to provide hints or real-time error analysis as you type. As this is written, RubyMine provides significant support for RBS files.

The Ruby interpreter could also use RBS information to optimize code generation, it seems as though more on that line of work is yet to come.

You may need to `gem install rbs` to get access to the RBS command line tools.

RBS offers its own command-line tools. These are generally proof of concept tools: `rbs list` gives you a list of classes and modules used by the application. The `rbs ancestors` and `rbs methods` tools both take the name of a class and provide what RBS knows about the ancestors or methods of that class:

```
$ rbs ancestors String
::String
::Comparable
::Object
::Kernel
::BasicObject
```

And the `rbs method` call takes a class and a method name and provides what RBS knows about that method:

```
rbs method String gsub
::String#gsub
  defined_in: ::String
  implementation: ::String
  accessibility: public
  types:
      (::Regexp | ::string pattern, ::string replacement) -> ::String
    | (::Regexp | ::string pattern, ::Hash[::String, ::String] hash) -> ::String
    | (::Regexp | ::string pattern) { (::String match) -> ::_ToS } -> ::String
    | (::Regexp | ::string pattern) -> ::Enumerator[::String, self]
```

There are other RBS commands that are not documented, and which presumably are either not expected to be in use, or are not yet complete.

The entire Ruby standard library has RBS files, so you can get type information about any method in that library.

Ruby also provides a tool called TypeProf, which can help you generate RBS files. To use it, add `gem "typeprof"` to the gemfile of the application you are working on and then `bundle install`.

What TypeProf does is take a Ruby file, which should be the top level file of your gem or application, and optionally an RBS file and it then spits out an RBS file. Here's what running TypeProf without the existing RBS file looks like for our gem: